

VNIVERSITAT DE VALÈNCIA

TEMA 2: Programación Funcional



VNIVERSITAT DE VALÈNCIA

Escola Tècnica Superior d'Enginyeria

Departament d'Informàtica



Contenidos



- Características de los lenguajes funcionales
- El lenguaje LISP
- Sintaxis



LISP



- Decada de los 50 (John McCarthy)
- Varias áreas de aplicación:
  - Robótica, IA, procesamiento de lenguaje natural, demostración automática de problemas
- LISP vs LISP Puro



Breve perspectiva del lenguaje LISP



- LISP (1958): Primer lenguaje funcional
  - Originalmente orientado a IA (LISt Processing)
  - Dialectos de LISP:
    - Scheme (1975)
    - COMMON LISP (1984)
  - Otros lenguajes funcionales:
    - ML
    - Miranda
    - Haskell



Principios de la Programación Funcional



- Uso del concepto matemático de función como estructura de control principal
- Estructuras de control:
  - Composición funcional
  - Construcción condicional
  - Recursividad
- Un programa consiste en una única expresión funcional que puede incluir llamadas a funciones



Composición funcional



- Python

```
def doble(x): return x * 2
def suma_tres(x): return x + 3

def compuesta(x):
    return suma_tres(doble(x))

print(compuesta(5))  # 13
```



Composición funcional



- Java

```
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        Function<Integer, Integer> doble = x -> x * 2;
        Function<Integer, Integer> sumaTres = x -> x + 3;

        Function<Integer, Integer> compuesta = doble.andThen(sumaTres);
        System.out.println(compuesta.apply(5)); // 13
    }
}
```



Características (II)



- Sin asignaciones

- Implicaciones:
  - Inmutabilidad
  - No existe el bucle como estructura de control



Inmutabilidad



- Python

```
tupla = (1, 2, 3)
# tupla[0] = 5  # Esto da error, las tuplas son inmutables
nueva_tupla = tupla + (4,)
print(nueva_tupla)  # (1, 2, 3, 4)
```



Inmutabilidad



- Java 9+:

```
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Integer> lista = List.of(1, 2, 3); // inmutable
        // lista.add(4); // Lanza UnsupportedOperationException
        System.out.println(lista);
    }
}
```



Características (III)



- Transparencia referencial:
  - El resultado de una función depende únicamente de los argumentos que se le pasan en la llamada
  - Implicaciones:
    - Inexistencia de efectos laterales
    - Muy adecuados para concurrencia





```
public class ClaseConEfectos{
  private boolean estado=false;
  public int calcula(int x) {
      estado=!estado;
      if (estado) return x+1;
      return 2*x;
  }
  public static void main(String[] args) {
     System.out.println(calcula(2)+" "+calcula(2));

  }
```



Características (IV)



- Funciones de primera clase:
  - Pueden pasarse como parámetro
- Implicaciones:
  - Programas y datos son “equivalentes”



Funciones de orden superior



- Python:

```
def aplicar_operacion(lista, operacion):
    return [operacion(x) for x in lista]

resultado = aplicar_operacion([1, 2, 3], lambda x: x * 2)
print(resultado)  # [2, 4, 6]
```



Funciones de orden superior



- Java 8+:

```
import java.util.*;
import java.util.function.Function;

public class Main {
    public static List<Integer> aplicarOperacion(List<Integer> lista,
                                                 Function<Integer, Integer> operacion)
   {
        List<Integer> resultado = new ArrayList<>();
        for (Integer x : lista) {
            resultado.add(operacion.apply(x));
        }
        return resultado;
    }

    public static void main(String[] args) {
        List<Integer> numeros = Arrays.asList(1, 2, 3);
        List<Integer> resultado = aplicarOperacion(numeros, x -> x * 2);
        System.out.println(resultado); // [2, 4, 6]
    }
}
```



Funciones anónimas



- Python:

```
cuadrado = lambda x: x**2
print(cuadrado(5))  # 25
```



Funciones anónimas



- Java:

```
Runnable r = () -> System.out.println("Hola desde una lambda en Java");
r.run();
```



Map, filter y reduce



- Python:

```
from functools import reduce

numeros = [1, 2, 3, 4]
dobles = list(map(lambda x: x * 2, numeros))
pares = list(filter(lambda x: x % 2 == 0, numeros))
suma = reduce(lambda a, b: a + b, numeros)

print(dobles)  # [2, 4, 6, 8]
print(pares)   # [2, 4]
print(suma)    # 10
```



Map, filter y reduce



- Java (con streams):

```
import java.util.*;
import java.util.stream.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> numeros = Arrays.asList(1, 2, 3, 4);

        List<Integer> dobles = numeros.stream()
                                      .map(x -> x * 2)
                                      .collect(Collectors.toList());

        List<Integer> pares = numeros.stream()
                                     .filter(x -> x % 2 == 0)
                                     .collect(Collectors.toList());

        int suma = numeros.stream()
                          .reduce(0, (a, b) -> a + b);

        System.out.println(dobles); // [2, 4, 6, 8]
        System.out.println(pares);  // [2, 4]
        System.out.println(suma);   // 10
    }
}
```



Características (V)



- Posibilidad de evaluación postergada (Lazy evaluation vs Eager evaluation – Haskell vs Lisp-)

```
x=(3*funcion(y)+7);
z=z+x;
```

- Implicaciones:
  - Evita cálculos y memoria innecesaria
  - Dificulta uso de excepciones y entrada/salida, por ejemplo



Carácterísticas (V)



- ¿Por qué no se hace en lenguajes operacionales?



Características (V)



- ¿Por qué no se hace en lenguajes operacionales?

Existen asignaciones!

No hay transparencia referencial!

```
x=(3*funcion(y)+7);
y=y+2;
z=y+x;
```



Sintaxis



- Basada en el concepto de s-expresión
- Todo programa/instrucción es una s-expresión
- Las s-expresiones pueden ser átomos o listas:
  - Un átomo puede ser un átomo numérico o simbólico
  - Una lista es una secuencia de s-expresiones separadas por espacios y delimitadas por paréntesis



Sintaxis



- Programas. Notación prefija y mediante listas para funciones:

  (f s_1 s_2 s_3 … s_n)

Cada s_i es una s-expresión

Ejemplo:

(* 6 (+ 2 3))



Evaluación de s-expresiones



- Dada una s-expresión S
  - Si S es un átomo numérico a, el resultado es a.
  - Si S es una lista se asume que tiene la forma (f s_1 s_2 s_3 … s_n):
    - se evalúa cada argumento s_i obteniendo valores (v_1, v_2, …v_n)
    - Posteriormente se calcula el resultado de aplicar la función con nombre *f* a esos valores. Nota: *f* no se evalúa.



Algunas funciones aritméticas



- (+ s_1 s_2 s_3 … s_n)
- (- s_1 s_2 s_3 … s_n)
- (* s_1 s_2 s_3 … s_n)
- (/ s_1 s_2 s_3 … s_n)
- (1+ s)
- (1- s)
- (MAX s_1 s_2 s_3 … s_n)
- (MIN s_1 s_2 s_3 … s_n)
- (MOD s_1 s_2)
- (FLOOR s_1) (CEIL s_1) ….

Cada s_i es una s-expresión que evalúa a un resultado numérico



Ejercicio



- Expresar:

  3+(5*6)-7

  (4*5*5)+5

  (2+1)*9

  ((-3*2)+16)/4

  2+3-6

  El máximo de los valores anteriores



Ejercicio - Respuestas



3+(5*6)-7	(- (+ 3 (* 5 6)) 7)
(4*5*5)+5	(+ (* 4 5 5) 5)
(2+1)*9	(* (+ 2 1) 9)
((-3*2)+16)/4	(/ (+ (* (- 3) 2) 16) 4)
2+3-6	(- (+ 2 3) 6)



Ejercicio - Respuestas



El máximo de los valores anteriores

(MAX (- (+ 3 (* 5 6)) 7)
(+ (* 4 5 5) 5)
(* (+ 2 1) 9)
(/ (+ (* (- 3) 2) 16) 4)
(- (+ 2 3) 6)
)



Ejercicio



- Calcula el resultado de las siguientes s-expresiones

- (+ 4 5 (- 4 3 (* 1 1)))
- (- (* (+ 3 4) (+ 1 4)))
- (* 1 (- 6 3) (- 8 5) (* 2 2) 2)
- (- (* 2 3))



Ejercicio - Respuestas



- (+ 4 5 (- 4 3 (* 1 1))) 9
- (- (* (+ 3 4) (+ 1 4))) -35
- (* 1 (- 6 3) (- 8 5) (* 2 2) 2) 72
- (- (* 2 3)) -6



Manipulación de listas (I)



- (LIST s_1 s_2 … s_n)
  - Devuelve una lista formada por los elementos especificados

```
> (LIST 5 4 5 6)
(5 4 5 6)
> (LIST (+ 1 2) 3 4)
(3 3 4)
```

Siempre se evalúan los argumentos y luego se aplica la función



Manipulación de listas (I)



- (CAR lista) Retorna el primer elemento de una lista
- (CDR lista) Retorna la cola de la lista



Manipulación de listas (I)



- (CAR lista) Retorna el primer elemento de una lista

- (CDR lista) Retorna la cola de la lista

> (CAR (7 3 2)) ¿Funcionaría?



Recordamos



- Dada una s-expresión S
  - Si S en un átomo numérico a, el resultado es a.
  - Si S es de la forma (f s_1 s_2 s_3 … s_n):
    - se evalúa cada argumento s_i obteniendo valores (v_1, v_2, …v_n)
    - Posteriormente se calcula el resultado de aplicar la función de nombre f a esos valores (f no se evalúa)



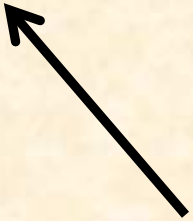
Manipulación de listas (I)



- (CAR lista) Retorna el primer elemento de una lista

- (CDR lista) Retorna la cola de la lista

> (CAR (7 3 2))



Evaluaría el argumento



QUOTE (‘) Y EVAL



- QUOTE (‘) se utiliza para evitar la evaluación del argumento
- EVAL tiene el efecto contrario: forzar la evaluación de un argumento

- Ejemplos:

```
> (QUOTE(* 3 3))          ó     ‘(* 3 3)
(* 3 3)
> (EVAL (QUOTE(* 3 3)))
9
```



Manipulación de listas (I)



- (CAR lista) Retorna el primer elemento de una lista
- (CDR lista) Retorna la cola de la lista
- (LENGTH lista) Retorna el tamaño de la lista
- Ejemplo:

  > (CAR '(7 3 2 1)) devuelve 7

  > (CDR '(7 3 2 1)) devuelve la lista (3 2 1)

  > (LENGHT '(7 3 2 1)) devuelve 4



Manipulación de listas (II)



- (CONS elemento lista)
  - Devuelve nueva lista cuyo primer elemento es elemento y cuya cola es lista
- (APPEND $lista_1$ $lista_2$ … $lista_n$)
  - Concatenación de listas
- (REVERSE lista)



Átomos especiales



- T (verdadero)
- NIL (falso, indefinido, lista vacía) – Crítica
- Valores numéricos

- Por convención, cualquier valor distinto de NIL denota verdadero



Predicados comprobación tipo



- (ATOM arg) – ¿Es un átomo? T y NIL lo son
- (LISTP arg) – ¿Es una lista?
- (NUMBERP arg) – ¿Es un número?



Pred. comprobación valor



- (ZEROP arg) – Devuelve T si arg es 0
- (MINUSP arg) – Devuelve T si arg es negativo
- (NULL arg) – Devuelve T si arg es NIL



Predicados comparación valores



- (EQUAL arg_1 arg_2)
- (MEMBER Elemento Lista)
- (< arg_1 arg_2 arg_3 … arg_n)
- (> arg_1 arg_2 arg_3 … arg_n)
- (= arg_1 arg_2 arg_3 … arg_n)



Predicados lógicos



- (NOT a)
- (AND a_1 a_2 a_3 … a_n)
  - En el caso de evaluar todos a verdadero, devuelve el último valor obtenido (el de más a la derecha)
- (OR a_1 a_2 a_3 … a_n)
  - En el caso de evaluar a verdadero, devuelve el primer valor distinto de NIL obtenido (el de más a la izquierda)



Estructura condicional



- (IF Condicion ResultadoSiT ResultadoSiNoT)



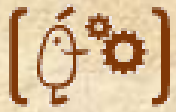
Estructura condicional



- (COND (Condicion$_1$ Resultado$_1$)
  (Condicion$_2$ Resultado$_2$)
  …
  (Condicion$_n$ Resultado$_n$))

Si ninguna condición evalúa a verdadero devuelve NIL

Para hacer condiciones exhaustivas, la Condicion$_n$ puede ser T



Definición de nuevas funciones



(DEFUN *nombre*(*parámetros*)

expresión resultado

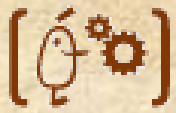
)



Definición de nuevas funciones



```
(DEFUN  factorial(N)
   (COND ((ZEROP N) 1)
                 (T (* N (factorial (- N 1))))
   )
)
```



Definición de nuevas funciones



```
(DEFUN  contarElementos(Lista)
   (COND ((NULL Lista) 0)
                (T (+ 1 (contarElementos (CDR Lista))))
     )
)
```



Definición de nuevas funciones



- ¿qué hace DEFUN?
  - DEFUN es una macro que añade una nueva entrada en la tabla de funciones



Definición de nuevas funciones



- Tabla de funciones
- Entradas (*nombre* -> *expresión lambda*)

```
(DEFUN nombre(parámetros)
     expresión resultado
 )
```

```
nombre -> (lambda (parámetros)
                expresión resultado
              )
```



Funcionamiento detallado



- Funcionamiento:
  - Tabla de funciones
    - Nombre -> expresión lambda
  - Evaluación de s-expresiones
    - Si hay una función con el nombre del primer elemento en la tabla de funciones (sin evaluar), se sustituye por la expresión lambda correspondiente
    - Se evalúa la expresión lambda
    - Funcionamiento expresiones lambda:
      - Se evalúan los argumentos
      - Binding (parámetros toman los valores de los argumentos, basado en su posición –tabla de símbolos-)
      - Evaluación -> Expresión resultado





- Las funciones lambda pueden usarse directamente (reemplazando al nombre de la función)
  - ((lambda (x) (+ 1 x)) 5)
  - (defun *plusone*(x) (+ 1 x))
    (*plusone* 5)



Llamadas a función



(*nombre* argumentos)

Donde *nombre* debe existir en la tabla

((lambda (*parámetros*) (*expresión resultado*)) argumentos)



Funciones como parámetro



- Queremos construir una función que reciba otra como parámetro

- Ejemplo: Aplicar a todos los elementos de una lista una determinada función (mapcar)



Funciones como parámetro



```
(defun aplicarAtodos(f l)
   (if (NULL l)
       NIL
       (cons (f (car l)) (aplicarAtodos f (cdr l)))
   )
)

(defun plus1(x) (1+ x))

(aplicarAtodos plus1 '(1 2 3))
```

?

correcto?



Funcionamiento detallado



- Funcionamiento:
  - Tabla de funciones
    - Nombre -> expresión lambda
  - Evaluación de s-expresiones
    - Si hay una función con el nombre del primer elemento en la tabla de funciones (sin evaluar), se sustituye por la expresión lambda correspondiente
    - Se evalúa la expresión lambda
    - Funcionamiento expresiones lambda:
      - Se evalúan los argumentos
      - Binding (parámetros toman los valores de los argumentos, basado en su posición –tabla de símbolos-)
      - Evaluación -> Expresión resultado





(defun plus1(x) (1+ x))

(aplicarAtodos plus1 '(1 2 3))

Se intentaría evaluar





```
(defun plus1(x) (1+ x))

(aplicarAtodos ‘plus1 ‘(1 2 3))

(defun aplicarAtodos(f l)
   (if (NULL l)
       NIL
       (cons (f (car l)) (aplicarAtodos f (cdr l)))
   )
)
```

correcto??





```
(defun plus1(x) (1+ x))

(aplicarAtodos ‘plus1 ‘(1 2 3))

(defun aplicarAtodos(f l)
   (if (NULL l)
       NIL
       (cons (f (car l)) (aplicarAtodos f (cdr l)))
   )
)
```

Los nombres de función no se evalúan



Funcionamiento detallado



- Funcionamiento:
  - Tabla de funciones
    - Nombre -> expresión lambda
  - Evaluación de s-expresiones
    - Si hay una función con el nombre del primer elemento en la tabla de funciones (sin evaluar), se sustituye por la expresión lambda correspondiente
    - Se evalúa la expresión lambda
    - Funcionamiento expresiones lambda:
      - Se evalúan los argumentos
      - Binding (parámetros toman los valores de los argumentos, basado en su posición –tabla de símbolos-)
      - Evaluación -> Expresión resultado



FUNCALL



Nombre y función son cosas distintas

(*nombre* argumentos)

((lambda (*parámetros*) (*expresión resultado*)) argumentos)

(funcall *función* argumentos)

(funcall *expresión lambda* argumentos)

(funcall *expresión que evalúa a nombre* argumentos)



FUNCALL



Nombre y función son cosas distintas

Para obtener la función a partir de su nombre:

#'nombre

(FUNCTION nombre)



FUNCALL



```
> (funcall #'+ 3 4)
7
```



APPLY



```
> (apply #'+ 3 4 '(3 4))
14
```



Funciones como parámetro



```
(defun aplicarAtodos(f l)
   (if (NULL l)
       NIL
       (cons (funcall f (car l))
             (aplicarAtodos f (cdr l)))
   )
)

(aplicarATodos #'plus1 '(1 2 3))
```



Paso de funciones como parámetro



```
> (defun menor (x y) (< x y))
> (defun mayor (x y) (> x y))

> (sort '(1 5 3 7 4 2) #'menor)
> (sort '(1 5 3 7 4 2) #'mayor)
> (sort '(1 5 3 7 4 2) (lambda (x y) (< x y)))
```

Siempre pasamos una función



Ejercicio



- Dadas las funciones f y g, y el valor x, compute f(g(x))

- Realiza la llamada para calcular la raíz cuadrada del valor absoluto de -9



Paso de funciones como parámetro



```
(defun composicion(f g x)
      (funcall f (funcall g x ))
)

(composicion #'sqrt #'abs -9)
```

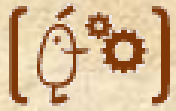


Paso de funciones como parámetro



```
> (defun plus1(x) (1+ x))
> (mapcar #'plus1 '(1 2 3 4))

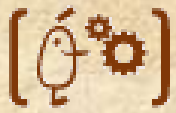
> (mapcar (lambda(x) (1+ x)) '(1 2 3 4))
```



Número de parámetros variable



- Intentemos imitar a la función +, llamándola suma y aceptando un número de parámetros variable



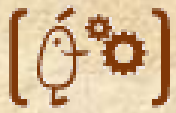
Número de parámetros variable



```
(defun suma(a &rest l)



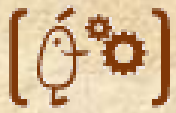
)
```



Número de parámetros variable



```
(defun suma(a &rest l)
   (if (NULL l)
         a
         (+ a (eval (cons 'suma l)))
    )
)
```



Número de parámetros variable

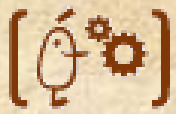


```
(defun suma(a &rest l)
   (if (NULL l)
         a
         (+ a (apply #'suma (car l) (cdr l)))  )
    )
)
```

Número de parámetros variable

```
(defun suma(a &rest l)
   (if (NULL l)
       a
       (+ a (apply #'suma (car l) (cdr l)) )
    )
)
```

¿Y por qué no (suma (car l) (cdr l))?

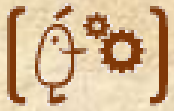


Número de parámetros variable



```
(defun
        a
        (+ a (apply #'suma (car l) (cdr l))  )
    )
)
```

¿Y por qué no
(funcall #'suma (car l) (cdr l))?



Número de parámetros variable



Otra alternativa interesante (y mucho más sencilla):

```
(defun suma(&rest l)
  (apply #'+ l)
)
```



Ejercicios - Retos



- Hacer una función que permita aplicar n funciones sobre dos números y devolver los resultados en una lista.

(aplicar 2 3 #'+ #'* (lambda (x y) (- x y)))

debería devolver

(5 6 -1)





```
(defun aplicar (x y &rest l)
    (if (NULL l)
        NIL
        (cons
           (funcall (car l) x y)
           (apply #'aplicar x y (cdr l))
        )
    )
)
```



Representación de tipos de datos



- Matrices
- Árboles